LAPORAN TUGAS

Tugas Besar 1

Mata kuliah	TI6053 – JARINGAN SYARAF TIRUAN
Dosen Pengampu	Matahari Bhakti Nendya, S.Kom., M.T,
Nama Kelompok	MasPras
Anggota Kelompok	1. 71230974 - Jimmy Christian 2. 71230987 - Prastha Pradipta Purusa 3. 71230988 - Calvin Putra Susanto
Deklarasi	Dengan ini kami menyatakan bahwa tugas ini merupakan hasil karya kelompok kami, tidak ada manipulasi data serta bukan merupakan plagiasi dari karya orang lain.

UNIVERSITAS KRISTEN DUTA WACANA
Fakultas Teknologi Informasi
Program Studi Informatika

Masalah dan Tujuan

Masalah didasari dari banyaknya faktor yang dapat mempengaruhi nilai ujian siswa seperti kebiasaan belajar, lingkungan keluarga, sekolah, fasilitas yang didapat, dan masih banyak lagi faktor lainnya yang hingga saat ini masih belum diketahui bobot pengaruhnya secara kuantitatif. Jika dilakukan analisis secara manual untuk menilai faktor - faktor yang mempengaruhi akan terasa berat.

Dari masalah tersebut diperlukan sebuah model yang dapat membantu melakukan analisis faktor - faktor yang dapat mempengaruhi nilai siswa. Maka dari itu, tujuan dari model ini adalah untuk :

- Mengidentifikasi faktor - faktor yang paling berpengaruh terhadap nilai ujian siswa
- Membangun model prediktif untuk memperkirakan nilai siswa berdasarkan faktor akademik, personal, dan sosioekonomi
- Memberi insight untuk intervensi (misal : jam belajar optimal, pentingnya tutoring)

Dataset

Dataset yang digunakan berasal dari kaggle, berikut link:
https://www.kaggle.com/datasets/lainguyn123/student-performance-factors/data

Dataset yang digunakan berisi mengenai beberapa faktor yang mempengaruhi exam score, berikut beberapa atribut yang terdapat pada dataset:
Hours_Studied, Attendance, Parental_Involvement, Access_to_Resources, Extracurricular_Activities, Sleep_Hours, Previous_Scores, Motivation_Level, Internet_Access, Tutoring_Sessions, Family_Income, Teacher_Quality, School_Type, Peer_Influence , Physical_Activity, Learning_Disabilities, Parental_Education_Level, Distance_from_Home , Gender , Exam_Score

Target Feature (Y) yang digunakan adalah Exam_Score, sedangkan atribut lainnya digunakan sebagai Input_Feature (X). Dikarenakan Target_Feature sudah jelas, maka lebih cocok digunakan metode supervised learning.

Artikel dan Code Pendukung

https://medium.com/@becaye-balde/perceptron-building-it-from-scratch-in-python-15716806ef64

Code Pendukung:

```python
def fit(self, X, y):

    # initializing the weights to 0
    self.w_ = np.zeros(1 + X.shape[1])
    self.errors_ = []

    print("Weights:", self.w_)

    # training the model n_iter times
    for _ in range(self.n_iter):
        error = 0

        # loop through each input
        for xi, y in zip(X, y):

            # 1. calculate ŷ (the predicted value)
            y_pred = self.predict(xi)

            # 2. calculate Update
            # update = η * (y - ŷ)
            update = self.eta * (y - y_pred)

            # 3. Update the weights
            # Wi = Wi + Δ(Wi)   where  Δ(Wi) = η * (y - ŷ) = update * Xi
            self.w_[1:] = self.w_[1:] + update * xi
            print("Updated Weights:", self.w_[1:])

            # Update the bias (Xo = 1)
            self.w_[0] = self.w_[0] + update

            # if update != 0, it means that ŷ != y
            error += int(update != 0.0)

        self.errors_.append(error)

    return self
```

Model Neuron dan Aturan Pembelajaran

Model Neuron

Setiap sampel memiliki:

- vektor fitur $x \in \mathbb{R}^d$, dengan d = jumlah fitur setelah encoding
- label $y \in \{-1, +1\}$, dengan +1 = Lulus

Input bersih (net input) :

$z = w^T x + b$

dimana :

w = vektor bobot, mengatur seberapa kuat setiap fitur mempengaruhi keputusan

b = bias, menggeser batas keputusan agar tidak harus melewati titik asal

Prediksi :

$\hat{y} = +1$ jika $z \geq 0$, dan $\hat{y} = -1$ jika $z < 0$

=> sample di satu sisi diprediksi Lulus, sisi lainnya Tidak Lulus

Aturan Pembelajaran

- Perceptron

  Pembaruan dilakukan per sampel menggunakan error dari keluaran berambang $\hat{y}$:

  - $\Delta = \eta\,(y - \hat{y})$
  - $w \leftarrow w + \Delta \cdot x$
  - $b \leftarrow b + \Delta$

Bobot hanya berubah pada saat klasifikasi, jumlah pembaharuan akibat salah klasifikasi dicatat per epoch. Jika data separable linear, jumlah ini akan mencapai nol. Jika tidak, jumlah akan terus berfluktuasi.

- Adaline
    - Menggunakan keluaran linear z untuk error $e = y - z$
    - Pembaruan batch gradient descent :
        - $w \leftarrow w + \eta \cdot X^T e / n$
        - $b \leftarrow b + \eta \cdot \text{mean}(e)$
    - Ambang kelas hanya digunakan saat prediksi

Fungsi Objektif

$$J = (1 / 2n) \Sigma (y_i - z_i)^2$$

dimana :

- $(y_i - z_i)$ adalah selisih antara label sebenarnya (−1 atau +1) dan keluaran linear model untuk sampel ke-i. Ini error sampel itu.
- Hasil pengurangan dikuadratkan supaya error negatif dan positif tidak saling menghapus, dan error besar dihukum lebih berat.
- Σ: untuk menjumlahkan semua n sampel latih.
- Dibagi 2n untuk dirata-ratakan, supaya nilainya tidak bergantung pada banyaknya data.

Penyiapan Data dan Rancangan Eksperimen

1. Penyiapan Data (Data Preparation)

Tahap penyiapan data bertujuan untuk memastikan bahwa dataset bersih, konsisten, dan siap digunakan dalam proses pelatihan model machine learning. Tahapan yang dilakukan meliputi:

a. Mengunduh dan Memuat Data

Dataset diunduh dari Google Drive menggunakan pustaka gdown, kemudian dimuat ke dalam struktur data DataFrame menggunakan pustaka Pandas untuk mempermudah proses pengolahan dan analisis data.

b. Penanganan Nilai Hilang (Missing Value Handling)

Identifikasi nilai yang hilang dilakukan pada kolom Teacher_Quality, Parental_Education_Level, dan Distance_from_Home. Nilai yang hilang pada kolom tersebut ditangani menggunakan metode imputasi modus, yaitu mengganti nilai kosong dengan nilai yang paling sering muncul pada masing-masing kolom.

c. Pembersihan Data (Data Cleaning)

Pembersihan data dilakukan dengan menghapus baris duplikat menggunakan fungsi drop_duplicates(). Langkah ini bertujuan untuk mengurangi redundansi data dan menjaga kualitas dataset yang digunakan dalam proses pemodelan.

d. Transformasi Variabel Target (Target Transformation)

Variabel target baru bernama Target dibuat berdasarkan nilai pada kolom Exam_Score. Penentuan label dilakukan menggunakan ketentuan berikut:

- Jika Exam_Score ≥ 68, maka siswa dikategorikan Lulus dengan label 1.
- Jika Exam_Score < 68, maka siswa dikategorikan Tidak Lulus dengan label -1.

Transformasi ini mengubah nilai ujian menjadi klasifikasi biner yang digunakan sebagai target prediksi oleh model.

e. Pembagian Data (Data Splitting)

Dataset dibagi menjadi 80% data latih (training set) dan 20% data uji (testing set). Pembagian dilakukan menggunakan metode stratified split untuk mempertahankan proporsi masing-masing kelas target pada kedua subset data. Dengan demikian, data latih dan data uji diharapkan dapat merepresentasikan distribusi kelas pada dataset secara lebih konsisten.

f. Rekayasa Fitur (Feature Engineering)

- Encoding fitur kategorikal: Fitur kategorikal diubah menjadi representasi numerik menggunakan metode One-Hot Encoding agar dapat diproses oleh algoritma machine learning.
- Standardisasi fitur: Fitur numerik dan hasil encoding diskalakan menggunakan StandardScaler agar memiliki rata-rata mendekati 0 dan standar deviasi mendekati 1.

Standardisasi sangat penting bagi algoritma Perceptron dan Adaline karena dapat membantu menyeimbangkan skala fitur, memperbaiki kestabilan proses pembelajaran, dan mendukung konvergensi model.

2. Rancangan Eksperimen (Experimental Design)

Eksperimen dirancang untuk menguji dan membandingkan kinerja dua algoritma klasifikasi linear dasar, yaitu Perceptron dan Adaline, dalam memprediksi kelulusan siswa berdasarkan karakteristik dan faktor-faktor yang berkaitan dengan performa akademik.

a. Algoritma yang Digunakan

1. Perceptron

Perceptron merupakan algoritma klasifikasi linear yang memperbarui bobot berdasarkan kesalahan prediksi. Pembaruan bobot dilakukan selama proses pembelajaran hingga model mencapai kondisi berhenti yang ditentukan atau batas jumlah iterasi (epoch) tercapai.

2. Adaline (Adaptive Linear Neuron)

Adaline merupakan algoritma pembelajaran linear yang menggunakan keluaran linear untuk menghitung kesalahan prediksi dan mengoptimalkan fungsi kerugian (loss function) berdasarkan Mean Squared Error (MSE). Pembaruan bobot dilakukan menggunakan metode Gradient Descent untuk meminimalkan nilai kesalahan selama proses pelatihan.

b. Metrik Evaluasi

Metrik utama yang digunakan adalah akurasi (accuracy score), yaitu persentase prediksi model yang sesuai dengan label sebenarnya pada data uji. Akurasi digunakan untuk membandingkan kemampuan Perceptron dan Adaline dalam mengklasifikasikan siswa ke dalam kategori lulus dan tidak lulus.

Selain akurasi, nilai loss atau kesalahan pembelajaran pada setiap epoch juga dianalisis untuk mengamati perkembangan proses pelatihan serta mengidentifikasi kemungkinan konvergensi atau divergensi model.

c. Alur Eksperimen

Proses eksperimen dilakukan melalui tahapan berikut:

1. Pelatihan model: Model Perceptron dan Adaline dilatih menggunakan fitur data latih yang telah melalui proses imputasi, encoding, dan standardisasi, yaitu X_train_scaled, serta label target y_train.

2. Prediksi data uji: Model yang telah dilatih digunakan untuk memprediksi label pada fitur data uji, yaitu X_test_scaled.

3. Evaluasi performa: Hasil prediksi dibandingkan dengan label sebenarnya pada data uji untuk memperoleh nilai akurasi masing-masing model.

4. Analisis proses pembelajaran: Perubahan nilai loss atau kesalahan pembelajaran sepanjang epoch diamati untuk mengetahui kestabilan pembelajaran dan kecenderungan konvergensi model.

5. Perbandingan hasil: Akurasi dan karakteristik proses pembelajaran kedua algoritma dibandingkan untuk mengetahui perbedaan kinerjanya dalam menyelesaikan permasalahan klasifikasi kelulusan siswa.

Hasil

Model	Learning Rate	Epoch	Standarisasi	Akurasi dan latih uji	Error/Loss Akhir
Perceptron	0.001	50	Ya	0.9357	333
Perceptron	0.01	50	Ya	0.9357	333
Perceptron	0.1	50	Ya	0.9357	333
Perceptron	0,01	50	Tidak	0.6256	1452
Adaline	0.001	50	Ya	0.9364	46
Adaline	0.01	50	Ya	0.9493	29
Adaline	0.1	50	Ya	0.9614	18
Adaline	0,01	50	Tidak	0.4554	1861190…..

Pembahasan dan Keterbatasan

Pembahasan :

- Data tidak separable sempurna, namun mendekati. Update perceptron turun dari 519 ke sekitar 300 lalu berfluktuasi, tidak pernah menyentuh nol. Akurasi uji 92,81% menunjukkan sebagian besar data terpisah, dengan irisan di dekat batas karena label dibuat dari ambang exam_score
- Perceptron menggunakan $y - \hat{y}$ (0 atau ±2) dan melakukan update per sampel, hanya saat salah. ADALINE menggunakan $y - z$ (kontinu)dan melakukan update batch tiap epoch, sehingga J dapat turun secara perlahan. Ambang hanya dipakai saat prediksi.
- Pengaruh η, epoch, standardisasi
    - Perceptron: dengan w awal nol dan urutan tetap, η hanya menskalakan bobot, sehingga hasil tiga η identik (sah, sesuai catatan panduan).
    - ADALINE: η kecil lambat, η besar bisa divergen.
    - Epoch: Perceptron tidak membaik setelah sekitar epoch 5.
    - Standardisasi: tanpanya, fitur berskala besar mendominasi gradien ADALINE.
- Akurasi yang sama tidak menjamin penggunaan proses yang sama juga. Keterbatasan yang ada dari kasus ini adalah label berasal dari buatan ambang, hanya berupa model linear, dan hanya melakukan satu split tanpa melakukan validasi ulang. Pengembangan yang mungkin dilakukan adalah melakukan validasi silang, label menggunakan ambang lain.

Kesimpulan

Model Perceptron menghasilkan akurasi uji sebesar 93,57% pada learning rate 0.001, 0.01, dan 0.1. Perubahan learning rate tidak mengubah akurasi akhir, tetapi mempengaruhi nilai bobot dan bias. Model belum mencapai error 0 karena jumlah pembaruan bobot masih berfluktuasi pada epoch terakhir.

Model Adaline menghasilkan akurasi 93,65% pada learning rate 0.001 dan mencapai akurasi tertinggi sebesar 96,14% pada learning rate 0.01. Sementara itu, learning rate 0.1 menyebabkan divergensi karena pembaruan yang terlalu besar. Berdasarkan hasil tersebut, Adaline dengan learning rate 0.01 menjadi model dengan akurasi tertinggi pada pengujian ini, meskipun evaluasi precision, recall, dan F1-score tetap diperlukan untuk menilai performanya secara lebih menyeluruh.

Ketika model dilatih menggunakan data asli tanpa StandardScaler, kedua algoritma mengalami penurunan performa. Perceptron hanya menghasilkan akurasi sekitar 57–62% karena fitur dengan rentang nilai besar, seperti Attendance, lebih dominan dalam perhitungan dan pembaruan bobot. Akibatnya, pembaruan bobot dapat menjadi tidak stabil, sehingga model kesulitan menemukan batas keputusan yang tepat. Jumlah kesalahan klasifikasi yang tinggi juga menunjukkan bahwa model belum mampu mengklasifikasikan data dengan baik.

Sementara itu, Adaline mengalami penurunan performa yang lebih parah, dengan akurasi sekitar 45% dan nilai loss yang membengkak hingga mencapai orde 10^150 atau lebih. Hal ini terjadi karena Adaline menggunakan perhitungan kesalahan kuadrat dan Gradient Descent, sehingga fitur bernilai besar dapat menghasilkan gradien yang sangat besar. Pembaruan bobot yang terlalu besar menyebabkan nilai parameter terus membesar dan proses pelatihan menjadi tidak stabil, bahkan dapat menghasilkan nilai tak hingga. Oleh karena itu, StandardScaler diperlukan untuk menyamakan skala fitur agar pembaruan bobot lebih stabil, sehingga performa Perceptron dan Adaline dapat meningkat.

Daftar Pustaka

Balde, B. (2023, Januari 28). *Perceptron: Building it from scratch in python*. Medium. https://medium.com/@becaye-balde/perceptron-building-it-from-scratch-in-python-15716806ef6

Lain, G. (n.d.). *Student performance factors*. Kaggle. Diambil dari https://www.kaggle.com/datasets/lainguyn123/student-performance-factors/data

MLearning.ai. (2021, 10 September). *Understanding ADALINE (Adaptive Linear Neuron)*. Medium. https://medium.com/mlearning-ai/understanding-adaline-da79ab8bbc5a

Lampiran:

```
def fit(self, X, y):

    # initializing the weights to 0
    self.w_ = np.zeros(1 + X.shape[1])
    self.errors_ = []

    print("Weights:", self.w_)

    # training the model n_iter times
    for _ in range(self.n_iter):
        error = 0

        # loop through each input
        for xi, y in zip(X, y):

            # 1. calculate ŷ (the predicted value)
            y_pred = self.predict(xi)

            # 2. calculate Update
            # update = η * (y - ŷ)
            update = self.eta * (y - y_pred)

            # 3. Update the weights
            # Wi = Wi + Δ(Wi)   where  Δ(Wi) = η * (y - ŷ) = update * Xi
            self.w_[1:] = self.w_[1:] + update * xi
            print("Updated Weights:", self.w_[1:])

            # Update the bias (Xo = 1)
            self.w_[0] = self.w_[0] + update

            # if update != 0, it means that ŷ != y
            error += int(update != 0.0)

        self.errors_.append(error)

    return self
```

Contoh Perhitungan Manual:

Bobot dan Bias awal yang digunakan adalah 0, jumlah fitur berubah menjadi 40 dikarenakan one hot encoding, namun karena terlalu banyak maka hanya dilakukan 1 uji menggunakan 40 atribut dan sisanya dilakukan dengan 2 atribut saja. Learning rate yang digunakan untuk perhitungan manual adalah 0.1, data yang digunakan adalah indeks ke 0 dengan label -1.

- **Perceptron**

  Sample 1

  Iterasi 1

  $z = w^T x + b$

  $z=((0 \times 0.505143) + (0 \times -1.545582) + (0 \times 0.657469) + (0 \times 1.118479) + (0 \times 1.211838) + (0 \times 0.039137) + (0 \times -0.635095) + (0 \times -0.504726) + (0 \times 0.981068) + (0 \times -0.650380)$

+ (0 × -0.505611) + (0 × 1.001704) + (0 × 1.213710) + (0 × -1.213710) + (0 × 1.967478)
+ (0 × -0.637443) + (0 × -1.011609) + (0 × 3.457373) + (0 × -3.457373) + (0 × 2.046083)
+ (0 × -0.829433) + (0 × -0.815853) + (0 × -0.655686) + (0 × 3.020682) + (0 × -1.22619)
+ (0 × -0.670473) + (0 × 0.670473) + (0 × -0.509444) + (0 × 1.232504) + (0 × -0.811356)
+ (0 × 0.339432) + (0 × -0.339432) + (0 × -0.655391) + (0 × 0.996789) + (0 × -0.496746)
+ (0 × -0.334908) + (0 × 1.512181) + (0 × -1.211806) + (0 × -0.850071)+(0 × 0.850071))
+ 0
z = 0
$\hat{y}$ = +1 dikarenakan z ≥ 0

$\Delta = \eta (y - \hat{y})$
$\Delta = 0.1 (-1 - (1))$
$\Delta = 0.1 (-2)$
$\Delta = -0.2$

Update bobot per atribut
(0 + (0.505143 × -0.2)), (0 + (-1.545582 × -0.2)), (0 + (0.657469 × -0.2)),
(0 + (1.118479 × -0.2)), (0 + (1.211838 × -0.2)), (0 + (0.039137 × -0.2)),
(0 + (-0.635095 × -0.2)), (0 + (-0.504726 × -0.2)), (0 + (0.981068 × -0.2)),
(0 + (-0.650380 × -0.2)),(0 + (-0.505611 × -0.2)), (0 + (1.001704 × -0.2)),
(0 + (1.213710 × -0.2)), (0 + (-1.213710 × -0.2)), (0 + (1.967478 × -0.2)),
(0 + (-0.637443 × -0.2)), (0 + (-1.011609 × -0.2)), (0 + (3.457373 × -0.2)),
(0 + (-3.457373 × -0.2)), (0 + (2.046083 × -0.2)), (0 + (-0.829433 × -0.2)),
(0 + (-0.815853 × -0.2)), (0 + (-0.655686 × -0.2)), (0 + (3.020682 × -0.2)),
(0 + (-1.226194 × -0.2)), (0 + (-0.670473 × -0.2)), (0 + (0.670473 × -0.2)),
(0 + (-0.509444 × -0.2)), (0 + (1.232504 × -0.2)), (0 + (-0.811356 × -0.2)),
(0 + (0.339432 × -0.2)), (0 + (-0.339432 × -0.2)), (0 + (-0.655391 × -0.2)),
(0 + (0.996789 × -0.2)), (0 + (-0.496746 × -0.2)), (0 + (-0.334908 × -0.2)),
(0 + (1.512181 × -0.2)), (0 + (-1.211806 × -0.2)), (0 + (-0.850071 × -0.2)),
(0 + (0.850071 × -0.2))

(0 + -0.1010286), (0 + 0.3091164), (0 + -0.1314938), (0 + -0.2236958),
(0 + -0.2423676), (0 + -0.0078274), (0 + 0.127019), (0 + 0.1009452),
(0 + -0.1962136), (0 + 0.130076), (0 + 0.1011222), (0 + -0.2003408),
(0 + -0.242742), (0 + 0.242742), (0 + -0.3934956), (0 + 0.1274886),
(0 + 0.2023218), (0 + -0.6914746), (0 + 0.6914746), (0 + -0.4092166),
(0 + 0.1658866), (0 + 0.1631706), (0 + 0.1311372), (0 + -0.6041364),
(0 + 0.2452388), (0 + 0.1340946), (0 + -0.1340946), (0 + 0.1018888),
(0 + -0.2465008), (0 + 0.1622712), (0 + -0.0678864), (0 + 0.0678864),
(0 + 0.1310782), (0 + -0.1993578), (0 + 0.0993492), (0 + 0.0669816),

(0 + -0.3024362), (0 + 0.2423612), (0 + 0.1700142), (0 + -0.1700142)

-0.1010286, 0.3091164, -0.1314938, -0.2236958, -0.2423676, -0.0078274, 0.127019, 0.1009452, -0.1962136, 0.130076, 0.1011222, -0.2003408, -0.242742, 0.242742, -0.3934956, 0.1274886, 0.2023218, -0.6914746, 0.6914746, -0.4092166, 0.1658866, 0.1631706, 0.1311372, -0.6041364, 0.2452388, 0.1340946, -0.1340946, 0.1018888, -0.2465008, 0.1622712, -0.0678864, 0.0678864, 0.1310782, -0.1993578, 0.0993492, 0.0669816, -0.3024362, 0.2423612, 0.1700142, -0.1700142

Update bias
b = 0 + (-0.2)
b = -0.2

Iterasi 2
$z = w^T x + b$
z = ((-0.1010286 × 0.505143) + (0.3091164 × -1.545582) + (-0.1314938 × 0.657469) + (-0.2236958 × 1.118479) + (-0.2423676 × 1.211838) + (-0.0078274 × 0.039137) + (0.127019 × -0.635095) + (0.1009452 × -0.504726) + (-0.1962136 × 0.981068) + (0.130076 × -0.650380) + (0.1011222 × -0.505611) + (-0.2003408 × 1.001704) + (-0.242742 × 1.213710) + (0.242742 × -1.213710) + (-0.3934956 × 1.967478) + (0.1274886 × -0.637443) + (0.2023218 × -1.011609) + (-0.6914746 × 3.457373) + (0.6914746 × -3.457373) + (-0.4092166 × 2.046083) + (0.1658866 × -0.829433) + (0.1631706 × -0.815853) + (0.1311372 × -0.655686) + (-0.6041364 × 3.020682) + (0.2452388 × -1.226194) + (0.1340946 × -0.670473) + (-0.1340946 × 0.670473) + (0.1018888 × -0.509444) + (-0.2465008 × 1.232504) + (0.1622712 × -0.811356) + (-0.0678864 × 0.339432) + (0.0678864 × -0.339432) + (0.1310782 × -0.655391) + (-0.1993578 × 0.996789) + (0.0993492 × -0.496746) + (0.0669816 × -0.334908) + (-0.3024362 × 1.512181) + (0.2423612 × -1.211806) + (0.1700142 × -0.850071) + (-0.1700142 × 0.850071)) + (-0.2)
z = −13,880108
$\hat{y}$ = -1 dikarenakan z < 0

$\Delta = \eta (y - \hat{y})$
$\Delta = 0.1 (-1 - (-1))$
$\Delta = 0.1 (0)$
$\Delta = 0$

Sample 2
Iterasi 1
$z = w^T x + b$

$z = (0 * 0{,}83790) + (0 * 0{,}52359) + 0$
$z = 0$
$\hat{y} = +1$ dikarenakan $z \geq 0$

$\Delta = \eta (y - \hat{y})$
$\Delta = 0.1 (-1 - (1))$
$\Delta = 0.1 (-2)$
$\Delta = -0.2$

Update bobot per atribut
$(0 + (0{,}83790 \times -0.2), (0 + (0{,}52359 \times -0.2))$
$(0 + (-0{,}16758)), (0 + (-0{,}104718))$
$((-0{,}16758), (-0{,}104718))$

Update bias
$b = 0 + (-0.2)$
$b = -0.2$

Iterasi 2
$z = w^T x + b$
$z = (-0{,}16758 * 0{,}83790) + (-0{,}104718 * 0{,}52359) - 0.2$
$z = -0{,}39523070162$
$\hat{y} = -1$ dikarenakan $z < 0$

$\Delta = \eta (y - \hat{y})$
$\Delta = 0.1 (-1 - (-1))$
$\Delta = 0.1 (0)$
$\Delta = 0$

- **Adaline (Menggunakan data sample)**

Data :

i	x1	x2	y
1	0,50514	-1,54558	-1
2	0,83790	0,52359	+1

  - Iterasi 1

$z = w^T x + b$
$z = 0$

$\hat{y} = +1$ dikarenakan $z \geq 0$

$e = (y - z) = (-1 - 0) = -1$
$J = (1 + 1) / (2 . 2) = ½ = 0.5$

Update:

- $\Sigma e_i x_{i1} = (-1)(0{,}50514) + (1)(0{,}83790) = 0{,}33276$
- $\Sigma e_i x_{i2} = (-1)(-1{,}54558) + (1)(0{,}52359) = 2{,}06917$
- $w_1 = 0 + 0{,}1 \cdot 0{,}33276 / 2 = 0{,}01664$
- $w_2 = 0 + 0{,}1 \cdot 2{,}06917 / 2 = 0{,}10346$
- mean(e) = 0, maka $b = 0 + 0 = 0$

- Iterasi 2

Z:

- $z_1 = (0{,}01664)(0{,}50514) + (0{,}10346)(-1{,}54558) = 0{,}00841 - 0{,}15991 = -0{,}15150$
- $z_2 = (0{,}01664)(0{,}83790) + (0{,}10346)(0{,}52359) = 0{,}01394 + 0{,}05417 = 0{,}06811$

E:

- $e_1 = -1 - (-0{,}15150) = -0{,}84850$
- $e_2 = 1 - 0{,}06811 = 0{,}93189$

J: $(0{,}71995 + 0{,}86842) / 4 = 0{,}39709$

Update:

- $\Sigma e_i x_{i1} = (-0{,}84850)(0{,}50514) + (0{,}93189)(0{,}83790) = -0{,}42861 + 0{,}78082 = 0{,}35221$
- $\Sigma e_i x_{i2} = (-0{,}84850)(-1{,}54558) + (0{,}93189)(0{,}52359) = 1{,}31143 + 0{,}48793 = 1{,}79936$
- $w_1 = 0{,}01664 + 0{,}1 \cdot 0{,}35221 / 2 = 0{,}03425$
- $w_2 = 0{,}10346 + 0{,}1 \cdot 1{,}79936 / 2 = 0{,}19343$
- mean(e) = $(-0{,}84850 + 0{,}93189) / 2 = 0{,}04169$, maka $b = 0 + 0{,}1 \cdot 0{,}04169 = 0{,}00417$